

Mathematical Formulation of Attention-Guided Reinforcement Learning

AttentionGuidedRL Repository

July 17, 2025

Abstract

This document provides a comprehensive mathematical formulation of the optimization objective implemented in the Attention-Guided Reinforcement Learning (AttentionGuidedRL) repository. The system trains a language model to autonomously guide its own training by sequencing key-value pairs from Wikipedia articles using reinforcement learning with policy gradients, multi-head attention mechanisms, and proximal policy optimization (PPO).

Contents

1	Introduction	1
2	Problem Formulation	1
2.1	State Space and Action Space	1
2.2	Trajectory and Episode Structure	1
3	Policy Definition	2
3.1	Vector Query Generation	2
3.2	Multi-Head Attention Similarity	2
4	Reward Function	3
5	Optimization Objective	3
5.1	Primary Objective Function	3
5.2	Return and Advantage Computation	4
6	Loss Function and Training Algorithm	4
6.1	PPO Loss Implementation	4
6.2	Step-Level Credit Assignment Details	5
6.3	KL Divergence Regularization	5
7	Training Algorithm	5
8	Implementation Details	5
8.1	Model Architecture	5
8.2	Training Configuration	6
8.3	Data Processing	6

9 Key Mathematical Properties	6
9.1 Convergence Properties	6
9.2 Attention Mechanism Properties	7
9.3 GRPO Baseline Properties	7
10 Conclusion	7

1 Introduction

The AttentionGuidedRL system implements a novel approach where a base language model (Llama-3.2-3B or GPT-2) learns to sequence its own training data through reinforcement learning. The model generates vector queries using attention mechanisms and selects key-value pairs to maximize learning progress, measured by conditional log probabilities of value tokens.

2 Problem Formulation

2.1 State Space and Action Space

Definition 1 (State Space). *At timestep t , the state s_t consists of:*

$$s_t = (c_t, \mathcal{K}_t^{available}) \quad (1)$$

where:

- $c_t \in \mathbb{Z}^{L_t}$ is the context sequence of length L_t
- $\mathcal{K}_t^{available} \subseteq \{1, 2, \dots, K\}$ is the set of available key indices

Definition 2 (Action Space). *The action space $\mathcal{A}_t$ at timestep t is the set of available key indices:*

$$\mathcal{A}_t = \mathcal{K}_t^{available} \quad (2)$$

2.2 Trajectory and Episode Structure

A trajectory τ consists of T steps:

$$\tau = \{(s_1, a_1, r_1), (s_2, a_2, r_2), \dots, (s_T, a_T, r_T)\} \quad (3)$$

where T is the number of key-value pairs per episode (NUM_KV_PAIRS in the code).

3 Policy Definition

3.1 Vector Query Generation

The policy π_θ operates through a vector query mechanism parameterized by LoRA adapter weights θ .

Definition 3 (Query Vector Generation). *Given context c_t , the query vector is generated as:*

$$q_t = \text{AvgPool}(\text{Embed}_\theta(c_t \oplus [\text{“Query”}])) \quad (4)$$

where:

- $Embed_\theta(\cdot)$ extracts embeddings from the second-to-last attention layer
- $\oplus$ denotes sequence concatenation
- $AvgPool$ averages over the sequence dimension
- $q_t \in \mathbb{R}^{d_{model}}$ where d_{model} is the model dimension

3.2 Multi-Head Attention Similarity

The system handles both standard Multi-Head Attention (MHA) and Grouped Query Attention (GQA).

Definition 4 (Head-wise Similarity Computation). *For each attention head $h \in \{1, 2, \dots, H\}$ and key k :*

$$sim_{t,k}^{(h)} = \frac{(q_t^{(h)})^T k_k^{(group(h))}}{\sqrt{d_{head}}} \cdot \frac{1}{\tau} \quad (5)$$

where:

- $q_t^{(h)} \in \mathbb{R}^{d_{head}}$ is the query for head h
- $k_k^{(group(h))} \in \mathbb{R}^{d_{head}}$ is the key for the corresponding group
- $group(h) = \lfloor h \cdot G/H \rfloor$ maps heads to groups (for GQA)
- G is the number of key-value groups
- H is the number of query heads
- $d_{head} = d_{model}/H$ is the head dimension
- $\tau = 1.0$ is the temperature parameter

Definition 5 (Per-Head Probability Distribution). *For each head h , the probability distribution over keys is:*

$$p_{t,k}^{(h)} = \frac{\exp(sim_{t,k}^{(h)})}{\sum_{k' \in \mathcal{K}_t^{available}} \exp(sim_{t,k'}^{(h)})} \quad (6)$$

Definition 6 (Policy Distribution). *The final policy distribution is obtained by averaging over heads:*

$$\pi_\theta(k|s_t) = \frac{1}{H} \sum_{h=1}^H p_{t,k}^{(h)} \quad (7)$$

4 Reward Function

Definition 7 (Step Reward). *The reward at step t is defined as the conditional log probability of value tokens:*

$$r_t = \begin{cases} \log p_\theta(v_t|c_t, k_t) - \log p_{\text{ref}}(v_t|c_t, k_t) & \text{if } \text{SUBTRACT_BASE_MODEL_LOGPROBS} \\ \log p_\theta(v_t|c_t, k_t) & \text{otherwise} \end{cases} \quad (8)$$

where:

- v_t are the value tokens at step t
- k_t are the selected key tokens at step t
- $p_\theta(\cdot)$ is the adapter model probability
- $p_{\text{ref}}(\cdot)$ is the reference model probability (base model without LoRA)

Definition 8 (Conditional Log Probability). *The conditional log probability is computed as:*

$$\log p_\theta(v_t|c_t, k_t) = \frac{1}{|v_t|} \sum_{i=1}^{|v_t|} \log p_\theta(v_{t,i}|c_t, k_t, v_{t,<i}) \quad (9)$$

where $v_{t,i}$ is the i -th token in the value sequence and $v_{t,<i}$ represents the preceding tokens.

5 Optimization Objective

5.1 Primary Objective Function

Theorem 1 (Trajectory Sampling with Step-Level Credit Assignment). *The system maximizes the following objective function:*

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=1}^T A_t(\tau) \cdot \log \pi_\theta(a_t|s_t) \right] - \beta \cdot D_{KL}(\pi_\theta || \pi_{\text{ref}}) \quad (10)$$

where:

- $\tau = (s_1, a_1, r_1, \dots, s_T, a_T, r_T)$ is a complete trajectory sampled from policy π_θ
- $A_t(\tau) = R_t(\tau) - b_t$ is the **trajectory-dependent advantage** at step t
- $R_t(\tau) = \sum_{t'=t}^T \gamma^{t'-t} r_{t'}$ are the discounted returns from step t forward in trajectory τ
- $b_t = \mathbb{E}_{\text{batch}}[R_t(\tau)]$ is the GRPO baseline (batch average of returns at step t)
- $\gamma = 0.99$ is the discount factor
- $\beta = 0.1$ is the KL penalty coefficient
- π_{ref} is the fixed reference policy (base model without LoRA)

Key Insight: We sample complete trajectories first, then assign step-specific credit. Each action a_t receives weight $A_t(\tau)$ that depends on the **actual future rewards** observed in that trajectory, not a trajectory-average.

5.2 Return and Advantage Computation

Definition 9 (Discounted Returns). *The return at timestep t is:*

$$R_t = \sum_{t'=t}^T \gamma^{t'-t} r_{t'} \quad (11)$$

Definition 10 (GRPO Advantage Estimation). *The system uses Group Relative Policy Optimization (GRPO) for advantage estimation:*

$$A_t(\tau^{(i)}) = R_t(\tau^{(i)}) - b_t \quad (12)$$

$$b_t = \frac{1}{B} \sum_{i=1}^B R_t(\tau^{(i)}) \quad (13)$$

where:

- $\tau^{(i)}$ is the i -th trajectory in the batch
- b_t is the per-timestep baseline computed as the batch average of returns at step t
- B is the batch size
- Each trajectory gets its own advantage based on its actual returns vs. the batch average

6 Loss Function and Training Algorithm

6.1 PPO Loss Implementation

Theorem 2 (Step-Level PPO Loss with Process Supervision). *The actual loss function implements step-level Proximal Policy Optimization where each action receives credit for its own contribution:*

$$\mathcal{L}(\theta) = - \sum_{t=1}^T [\min(\rho_t(\theta) \cdot A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A_t)] + \beta \cdot D_{KL}(\pi_\theta || \pi_{\text{ref}}) \quad (14)$$

where each step t contributes independently:

- $A_t = R_t - b_t$ is the **step-specific advantage** (return from step t forward minus baseline)
- $\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}$ is the **step-specific probability ratio**
- a_t is the key selection action at step t given context s_t
- $\epsilon = 0.2$ is the PPO clipping parameter
- $\text{clip}(x, a, b) = \max(a, \min(x, b))$
- KL divergence is computed against the fixed reference policy π_{ref}

Process Supervision Interpretation: Each action a_t (key selection) receives credit proportional to its step-specific advantage A_t , enabling proper credit assignment where early selections (affecting more future rewards) receive higher weight than later selections.

6.2 Step-Level Credit Assignment Details

Definition 11 (Step-Level Gradient Decomposition). *The policy gradient decomposes naturally into step-level contributions:*

$$\nabla_{\theta} \mathcal{J}(\theta) = \sum_{t=1}^T \mathbb{E} [A_t \cdot \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] \quad (15)$$

This formulation ensures:

1. **Proper Temporal Credit:** Action a_1 (first key selection) affects rewards $\{r_1, r_2, \dots, r_T\}$, so $A_1 = \sum_{t'=1}^T \gamma^{t'-1} r_{t'} - b_1$
2. **Diminishing Influence:** Action a_T (last key selection) only affects r_T , so $A_T = r_T - b_T$
3. **Independent Assessment:** Each action’s contribution is evaluated against its own expected value, not averaged across the trajectory

Comparison with Trajectory-Level Formulation:

- **Trajectory-Level:** $\mathcal{L} = -\bar{A} \sum_{t=1}^T \log \pi_{\theta}(a_t | s_t)$ where $\bar{A} = \frac{1}{T} \sum_{t=1}^T A_t$
- **Step-Level:** $\mathcal{L} = -\sum_{t=1}^T A_t \log \pi_{\theta}(a_t | s_t)$ (our formulation)

The step-level formulation provides **finer-grained supervision** where early actions naturally receive higher gradients due to their broader impact on trajectory outcomes.

6.3 KL Divergence Regularization

Definition 12 (KL Divergence Term). *The KL divergence regularization is computed as:*

$$D_{KL}(\pi_{\theta} || \pi_{ref}) = \sum_{t=1}^T \sum_{k \in \mathcal{K}_t^{available}} \pi_{\theta}(k | s_t) \log \frac{\pi_{\theta}(k | s_t)}{\pi_{ref}(k | s_t)} \quad (16)$$

This is implemented using PyTorch’s `F.kl_div` with `log_target=True` for numerical stability. The reference policy π_{ref} is the fixed base model without LoRA, providing a stable regularization target.

7 Training Algorithm

8 Implementation Details

8.1 Model Architecture

- **Base Models:** Llama-3.2-3B or GPT-2
- **LoRA Configuration:** rank=8, $\alpha=16$, dropout=0.0
- **Precision:** bfloat16 for computation, float32 for similarity calculations
- **Context Window:** Up to model maximum (2048 for GPT-2, 4096+ for Llama)

Algorithm 1 Attention-Guided RL Training

Require: Base model, tokenizer, Wikipedia dataset

Require: Hyperparameters: γ, β, ϵ , learning rate, batch size

```
1: Initialize LoRA adapter parameters  $\theta$ 
2: Initialize reference model  $\pi_{\text{ref}}$  (base model without LoRA, fixed)
3: Initialize old model  $\pi_{\text{old}} \leftarrow \pi_{\theta}$ 
4: Initialize optimizer (AdamW with learning rate  $5 \times 10^{-4}$ )
5: for episode = 1 to NUM_EPISODES do
6:   Sample Wikipedia article and extract key-value pairs
7:   Generate trajectory  $\tau$  using current policy  $\pi_{\theta}$ 
8:   Compute rewards  $\{r_t\}_{t=1}^T$  using Equation (7)
9:   Compute returns  $\{R_t\}_{t=1}^T$  using Equation (10)
10:  Compute advantages  $\{A_t\}_{t=1}^T$  using GRPO (Equation 11-12)
11:  Compute PPO loss  $\mathcal{L}(\theta)$  using Equation (13) with KL vs  $\pi_{\text{ref}}$ 
12:  Update parameters:  $\theta \leftarrow \theta - \nabla_{\theta} \mathcal{L}(\theta)$ 
13:  if episode mod BASELINE_UPDATE_FREQUENCY == 0 then
14:    Update old model:  $\pi_{\text{old}} \leftarrow \pi_{\theta}$  (for PPO ratios only)
15:  end if
16: end for
```

8.2 Training Configuration

- **Optimizer:** AdamW with learning rate 5×10^{-4}
- **Batch Size:** 4 trajectories per update
- **Gradient Clipping:** Norm clipped to 1.0
- **Episodes:** 10,000 total training episodes
- **Checkpoint Frequency:** Every 100 episodes

8.3 Data Processing

- **Dataset:** Wikipedia 2022-03-01 English subset
- **Token Counts:** 10 tokens per key, 10 tokens per value
- **Trajectory Length:** Up to 10 key-value pairs per episode
- **Prefixes:** "Key: " and "Value: " for context building

9 Key Mathematical Properties

9.1 Convergence Properties

Lemma 1 (Policy Improvement). *Under the PPO clipped objective with appropriate clipping parameter ϵ , the policy improvement is guaranteed to be conservative, preventing destructively large updates.*

9.2 Attention Mechanism Properties

Lemma 2 (Multi-Head Averaging). *The averaging operation over attention heads in Equation (6) preserves the probability distribution property:*

$$\sum_{k \in \mathcal{K}_t^{\text{available}}} \pi_\theta(k|s_t) = 1 \quad (17)$$

9.3 GRPO Baseline Properties

Theorem 3 (Unbiased Advantage Estimation). *The GRPO baseline provides unbiased gradient estimates:*

$$\mathbb{E}[A_t] = 0 \quad \text{for each timestep } t \quad (18)$$

This property holds by construction since $A_t = R_t - \mathbb{E}[R_t]$ where the expectation is over the batch.

10 Conclusion

This mathematical formulation describes a sophisticated reinforcement learning system that combines:

1. **Vector-based policy:** Using attention layer embeddings for query generation
2. **Multi-head attention similarity:** Handling both MHA and GQA architectures
3. **Step-level PPO optimization:** With proper temporal credit assignment through step-specific advantages
4. **GRPO advantage estimation:** For variance reduction without additional parameters
5. **Process supervision:** Where each action receives credit proportional to its impact on future outcomes
6. **Self-supervised learning:** Where the model learns to sequence its own training data

The key innovation is the **step-level formulation** that ensures proper credit assignment: early key selections (which affect more future rewards) naturally receive higher gradient magnitudes than later selections. This creates a natural curriculum where the model learns to prioritize the quality of early decisions while still optimizing the entire sequence.

The system represents a novel approach to active learning where the model’s attention mechanisms directly drive the selection of training examples, creating a feedback loop between the model’s internal representations and its learning curriculum.

References

- [1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347.
- [2] Shazeer, N. (2019). *Fast transformer decoding: One write-head is all you need*. arXiv preprint arXiv:1911.02150.
- [3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models*. arXiv preprint arXiv:2106.09685.